

Dijkstra最短路算法

Dijkstra模板

P4779 【模板】单源最短路径（标准版）

- 题号: P4799
- 链接: [题目链接](#)
- 算法类型:最短路
- AC 代码:

```
void solve()
{
    ll n,m,s;
    cin>>n>>m>>s;
    //前面是点,后面是距离
    vector<vector<pair<int,ll>>> mp(n+1);
    rep(i,0,m-1)
    {
        int u,v,w;
        cin>>u>>v>>w;
        mp[u].push_back({v,w});
        // mp[v].push_back({u,w});这道题是有向边所以不要入两次!!
    }

    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>>pos ;
    vector<ll> dist(n+1,LINF);
    dist[s]=0;
    pos.emplace(0,s);

    while(!pos.empty())
    {
        auto [nowd,nowp] = pos.top();
        pos.pop();
        if(nowd>dist[nowp])continue;
        for(auto [nextp,nextd]:mp[nowp])
        {
            if(dist[nextp]>dist[nowp]+nextd)
            {
                dist[nextp]=dist[nowp]+nextd;
                pos.emplace(dist[nextp],nextp);
            }
        }
    }
    rep(i,1,n)
    {
        cout<<dist[i]<<' ';
    }
}
```

- **注意事项:**
 - 注意Dijkstra算法用最小堆优化可以时间复杂度最低
 - Dijkstra算法只能处理**非负权路径问题**
 - 为什么不用队列? : 贪心最快, 如果是菊花图复杂度会退化到 nm
 - 含负权路用什么算法? 用队列

Tarjan

Tarjan是什么?

- $O(V + E)$, 其中 V 是节点数, E 是边数
- 实现: stack dfs
- 讨论有向图的连通性
- Tarjan 算法是找出割点、桥和双连通分量的标准算法。

Tarjan怎么用?

- 对于每个节点 u 我们需要
 - $dfn[u]$ 访问顺序
 - $low[u]$ 判断 u 是否属于某个SCC
- 当

$$dfn[u] = low[u]$$

的时候 u 是这个SCC的根节点

- 只有当一个节点确定属于某个 SCC 时, 它才会被弹出栈。
- $in_stack[u]$ (或 $vis[u]$): 一个布尔数组, 标记节点 u 是否在 stack 中。//换名字以免和遍历算法重名
- 我们在根节点上面对他所领导的强连通分量做统计
- dfn 值是用来衡量 连通性 的标准。
- **tarjan模板**

```
const int MAXN = 100005;

// =====
// 核心数据结构和变量
// =====

// 图的邻接表表示
vector<int> adj[MAXN];

// 计时器, 用于给节点分配 dfn
int timer = 0;
```

```

// dfn[u]: 节点 u 的 DFS 访问次序 (发现时间)
int dfn[MAXN];

// low[u]: 节点 u 及其子树中的节点能追溯到的最早的 dfn
int low[MAXN];

// stack: 存储当前正在搜索路径上且未确定 SCC 的节点
stack<int> s;

// in_stack[u]: 标记节点 u 是否在栈中
bool in_stack[MAXN];

// scc_count: 强连通分量的数量
int scc_count = 0;

// scc_id[u]: 节点 u 所属的强连通分量的编号
// 如果需要进行“缩点”, 这个数组是关键
int scc_id[MAXN];

// =====
// Tarjan 算法主体
// =====

/**
 * @brief Tarjan算法寻找强连通分量
 * @param u 当前正在访问的节点
 */
void Tarjan(int u) {
    // 1. 初始化: 设置 dfn 和 low
    dfn[u] = low[u] = ++timer;

    // 2. 入栈: 将节点 u 压入栈并标记
    s.push(u);
    in_stack[u] = true;

    // 3. 遍历 u 的所有邻接点 v
    for (int v : adj[u]) {
        if (dfn[v] == 0) {
            // 情况 A: v 未被访问 (树枝边)
            Tarjan(v);

            // 回溯时: 用子节点的 low 更新父节点的 low
            // low[u] = min(low[u], low[v])
            low[u] = min(low[u], low[v]);
        } else if (in_stack[v]) {
            // 情况 B: v 已被访问且仍在栈中 (回边/横叉边, 但 v 在栈中)
            // 用 v 的 dfn 更新 u 的 low
            // low[u] = min(low[u], dfn[v])
            // 注意: 这里是 dfn[v] 而非 low[v], 因为 low[v] 可能跨越了 u 所在的 SCC
            low[u] = min(low[u], dfn[v]);
        }
    }

    // 4. 判断是否找到 SCC 根

```

```

// 如果 dfn[u] == low[u], 则 u 是一个 SCC 的根
if (dfn[u] == low[u]) {
    scc_count++;
    // 5. 出栈: 将 SCC 中的所有节点弹出
    while (true) {
        int current_node = s.top();
        s.pop();

        // 标记节点已不在栈中
        in_stack[current_node] = false;

        // 分配 SCC 编号
        scc_id[current_node] = scc_count;

        // 直到弹出根节点 u
        if (current_node == u) {
            break;
        }
    }
}
}

调用: // 遍历所有节点, 确保所有连通块都被搜索到
for (int i = 1; i <= N; ++i) {
    if (dfn[i] == 0) {
        Tarjan(i);
    }
}

```

P3916 图的遍历(运用tarjan算法)

- 题号: P3916
- 链接: [题目链接](#)
- 算法类型: Tarjan
- AC 代码:

```

const int MAXN = 100005;

vi dfn(MAXN);
vi low(MAXN);

stack<int> stk;

vector<bool> in_stk(MAXN, 0);
int scc = 0; // scc编号
int scc_id[MAXN];

vi nwempmx(MAXN);
vi dp(MAXN);
int timer = 0;

```

```
void Tarjan(int u, const vector<vi> &mp)
{
    dfn[u] = low[u] = ++timer;

    stk.push(u);
    in_stk[u] = 1;

    for (auto hsh : mp[u])
    {
        if (!dfn[hsh])
        {
            Tarjan(hsh, mp);
            low[u] = min(low[u], low[hsh]);
        }
        else if (in_stk[hsh])
        {
            low[u] = min(low[u], dfn[hsh]);
        }
    }

    if (dfn[u] == low[u])
    {
        scc++;
        int mxscc = 0;
        while (1)
        {
            auto node = stk.top();
            stk.pop();
            in_stk[node] = 0;
            scc_id[node] = scc;
            mxscc = max(mxscc, node);
            if (node == u)
                break;
        }
        nwempmx[scc] = mxscc;
        dp[scc] = mxscc;
    }
}

int dfs(int a, const vector<vi> &newmp)
{
    if (dp[a] != nwempmx[a])
    {
        return dp[a];
    }
    int k = nwempmx[a];
    for (int hsh4 : newmp[a])
    {
        k = max(k, dfs(hsh4, newmp));
    }
    dp[a] = k;
    return dp[a];
}
```

```
}

```